



QueueMind
FLOW INTELLIGENCE

Intelligent Canteen Flow Optimizer

Predict bottlenecks before they happen - then tell staff exactly what to do.

Working prototype presentation: problem, algorithm, interface, architecture, explainability, implementation and roadmap.

LIVE RUSH-HOUR SCENARIO

HIGH RISK

70 queued + 45 arriving in 15 min

4

Recommended counters

+35

Extra portions

14 → 7

Wait time (min)

Transparent rules

No external libraries or backend are required for the self-contained HTML demo.

What QueueMind actually does

A decision-support system for canteen operations, built around a 15-minute forecast horizon.

01

Sees the operating state

Captures the current queue, expected arrivals, food stock, service speed and active counters.

02

Forecasts pressure

Computes demand versus service capacity, projects queue movement and classifies crowd risk.

03

Recommends action

Outputs the minimum counters needed, extra portions to prepare and expected wait-time improvement.

CORE IDEA

QueueMind converts operational data into a simple staff decision.

Instead of reacting after a queue becomes unmanageable, the system warns early and recommends the staffing and food-preparation response.

PROBLEM

Why a canteen needs predictive flow management

Peak periods create a coordination problem: people, service capacity and food availability change at different speeds.



Long queues

Students arrive in bursts; staff often see the overload only after it forms.



Uneven counter load

Too few counters cause congestion; too many waste staff capacity.



Food mismatch

Preparing too little causes stock-outs; over-preparing increases waste.



Reactive decisions

Staff usually rely on visual judgement rather than a quantified 15-minute outlook.

Goal: move from reactive → predictive

From canteen inputs to an operational action

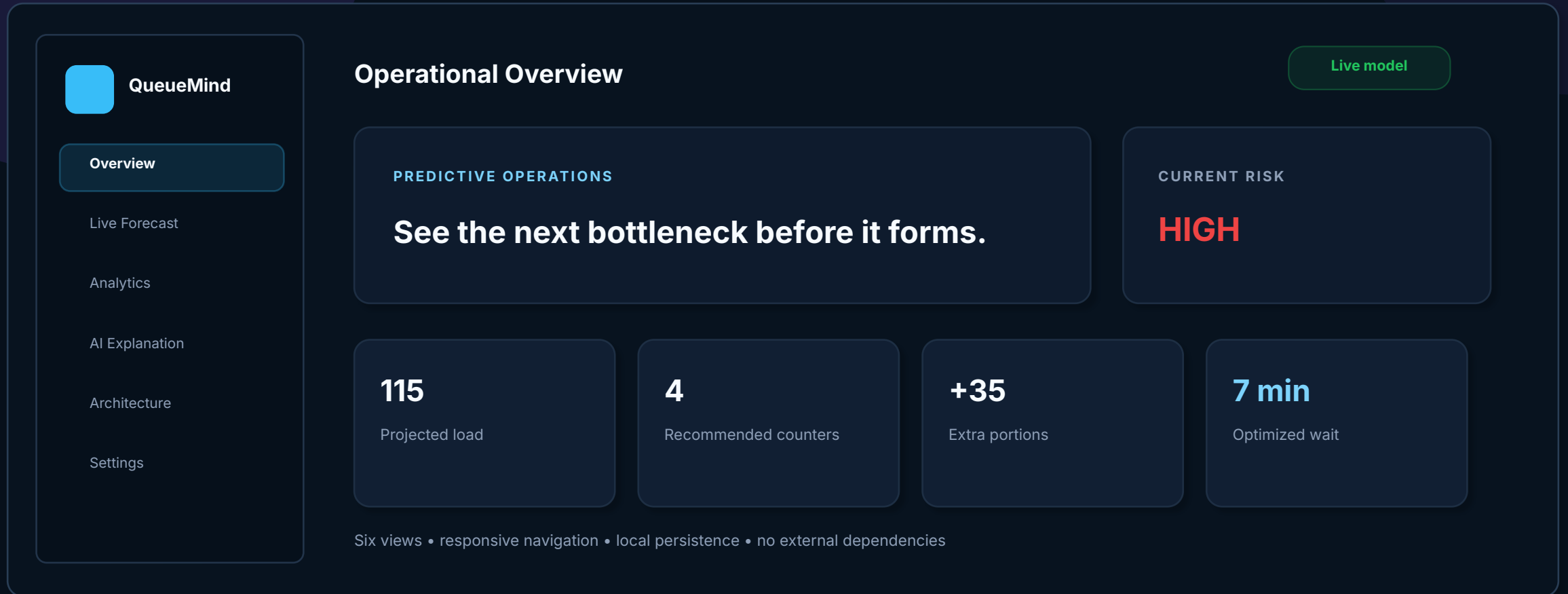
The prototype follows a simple five-step pipeline. Every step is visible and explainable.



The current working model uses deterministic equations for the forecast and rule-based decisions. Historical machine learning can be added later without changing the decision layer.

A real multi-view product experience

The single HTML file behaves like a multi-page application: navigation changes the workspace without reloading or requiring internet access.



INPUTS

Five variables define the operating state

Each input has a direct operational meaning and is adjustable live in the working model.



Current queue length

How many students are already waiting now.

0 - 200 students



Expected arrivals

Students predicted/estimated to arrive in the next 15 minutes.

0 - 150 students



Food available

Ready portions currently available for service.

0 - 250 portions



Service rate

Average students served per minute per counter.

1 - 10 / min / counter



Active counters

How many service counters are currently operating.

1 - 6 counters

Important: "Expected arrivals" is manually supplied in this demo. In production, it can come from historical patterns, class timetables, entry sensors or a trained arrival-prediction model.

The core forecasting logic

The present prototype is transparent operational intelligence: deterministic formulas + rule-based decisions, not a hidden trained model.

1. Total 15-minute demand

$\text{totalLoad} = \text{currentQueue} + \text{expectedArrivals}$

How many students the canteen must deal with.

4. Recommended counters

$\text{ceil}(\text{totalLoad} \div (\text{serviceRate} \times 15))$

Smallest whole counter count able to cover demand.

2. Current service capacity

$\text{capacity} = \text{counters} \times \text{serviceRate} \times 15$

How many students the active counters can clear.

5. Approx. wait time

$\text{queue} \div (\text{counters} \times \text{serviceRate})$

Simple queue-clearing estimate in minutes.

3. Food deficit

$\text{deficit} = \max(0, \text{totalLoad} - \text{foodStock})$

Extra portions needed to avoid stock-out.

6. Queue at time t

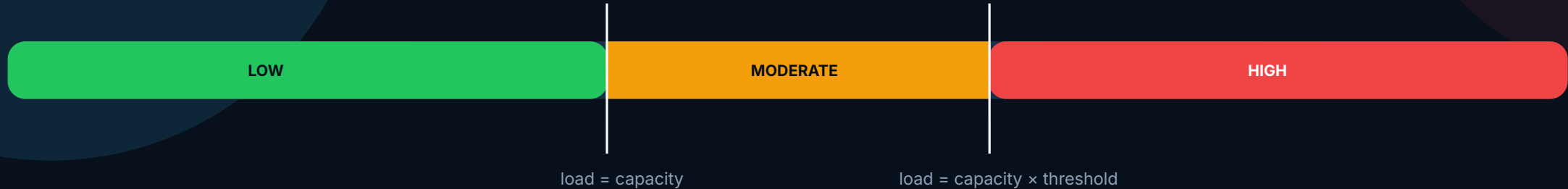
$\max(0, \text{queue} + \text{arrivalRate} \times t - \text{serviceRateTotal} \times t)$

Projects the queue curve at 0, 3, 6, 9, 12 and 15 minutes.

Deterministic & explainable

How crowd risk is classified

Demand is compared with current 15-minute capacity. The high-risk threshold is configurable in the working model.



LOW RISK

$\text{totalLoad} \leq \text{currentCapacity}$

Flow can be cleared with current staffing.

MODERATE RISK

$\text{currentCapacity} < \text{totalLoad} \leq \text{capacity} \times \text{threshold}$

Queue accumulation is likely.

HIGH RISK

$\text{totalLoad} > \text{capacity} \times \text{threshold}$

Demand materially exceeds current capability.

How QueueMind decides how many counters to open

The algorithm chooses the smallest whole number of counters whose 15-minute capacity covers projected demand.

FORMULA

```
recommendedCounters =  
ceil(totalLoad / (serviceRate × 15))
```

Why ceil()? You cannot open 3.07 counters. The system rounds upward so the recommendation is operationally feasible and has enough capacity.

Minimum sufficient staffing

RUSH-HOUR EXAMPLE

Projected load 115 students

One counter clears 37.5 students / 15 min

$115 \div 37.5$ 3.07 counters

Rounded upward **4 counters**

With 2 active counters, QueueMind recommends opening +2.

Demand forecasting becomes a stock decision

QueueMind compares projected 15-minute demand with ready portions and turns the gap into an immediate kitchen action.

STOCK EQUATION

`foodDeficit = max(0, totalLoad - foodAvailable)`

115

projected portions needed

– 80

currently available

= 35

extra portions

Kitchen alert: prepare +35

WHY IT MATTERS

01

Prevent stock-outs during surges

02

Avoid unnecessary large batches

03

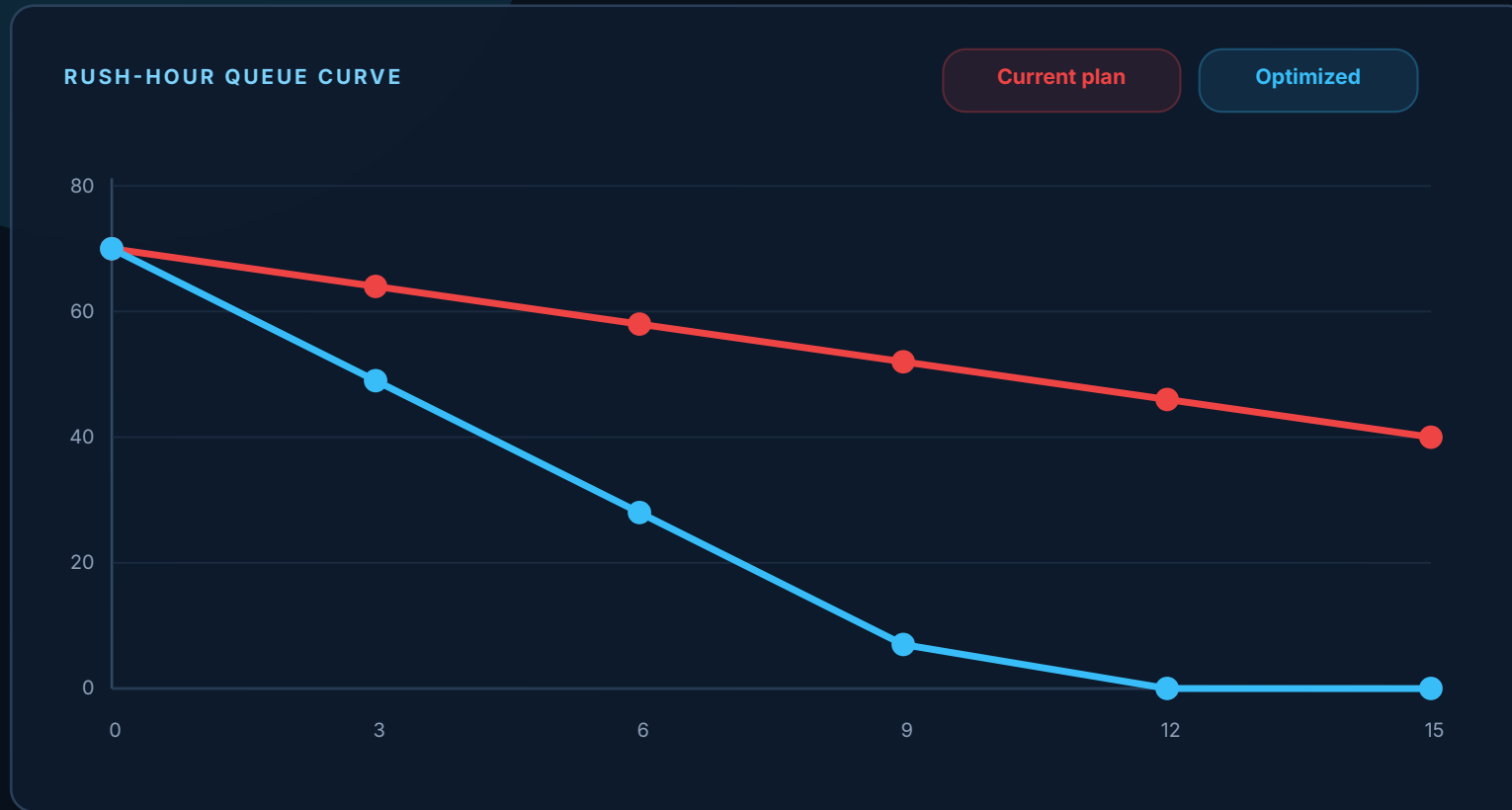
Synchronize kitchen prep with service demand

04

Create an auditable reason for each food alert

What happens over the next 15 minutes

The model compares the queue trajectory with current staffing versus the recommended counter plan.



RESULT

14 min

Approx. current wait

7 min

Approx. optimized wait

At minute 15, current staffing leaves ~40 students in queue; the recommended 4-counter plan clears the modeled queue by around minute 12.

Every recommendation includes its reasoning

The interface exposes a decision trace so users can see exactly how the output was produced.

01

Estimate total demand

70 current + 45 arriving = 115 projected students.

02

Measure capacity

2 counters $\times$ 2.5/min $\times$ 15 = 75 students.

03

Classify risk

$115 > 75 \times 1.3$, therefore HIGH RISK.

04

Choose staffing

$\text{ceil}(115 \div 37.5) = 4$ counters.

05

Check food

80 stock vs 115 demand = +35 portions needed.

HUMAN-READABLE ACTION

HIGH RISK**Open 2 additional counters now.****Prepare 35 extra portions.**

Projected wait improves from ~14 min to ~7 min with the recommended setup.

No black box

Turning forecast runs into operational learning

Saved runs are stored locally and can be compared to understand recurring demand patterns and model behavior.

01

Runs saved

Forecast history with timestamp and scenario inputs.

02

Risk distribution

Count of low, moderate and high-risk runs.

03

Wait-time trend

Current versus optimized wait across runs.

04

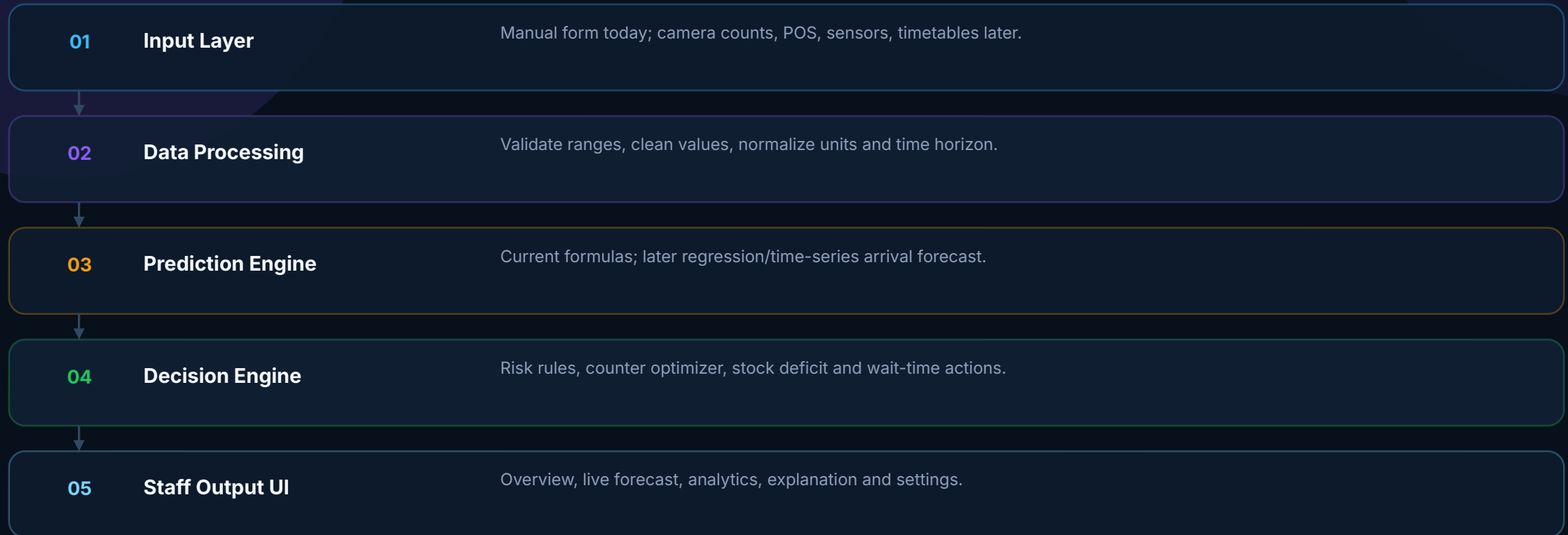
Capacity comparison

Projected load versus available service capacity.

Implementation detail: forecast history and settings are persisted with browser localStorage. This keeps the demo self-contained; a real deployment would write runs to a backend database for multi-user analytics.

Five-layer system design

The interface already separates data intake, processing, forecasting, decisions and staff presentation - making future backend integration straightforward.



Modular: UI can stay while the prediction layer evolves

What the current working model is built with

The delivered model is deliberately portable: one HTML file, no server setup and no third-party runtime dependency.

HTML5

Application structure and semantic views.

CSS3

Dark glass interface, responsive layout, controls and animations.

Vanilla JavaScript

Simulation logic, navigation, rendering and interaction.

Inline SVG

Queue projection and analytics charts without Chart.js.

localStorage

Persist settings and saved forecast runs in the browser.

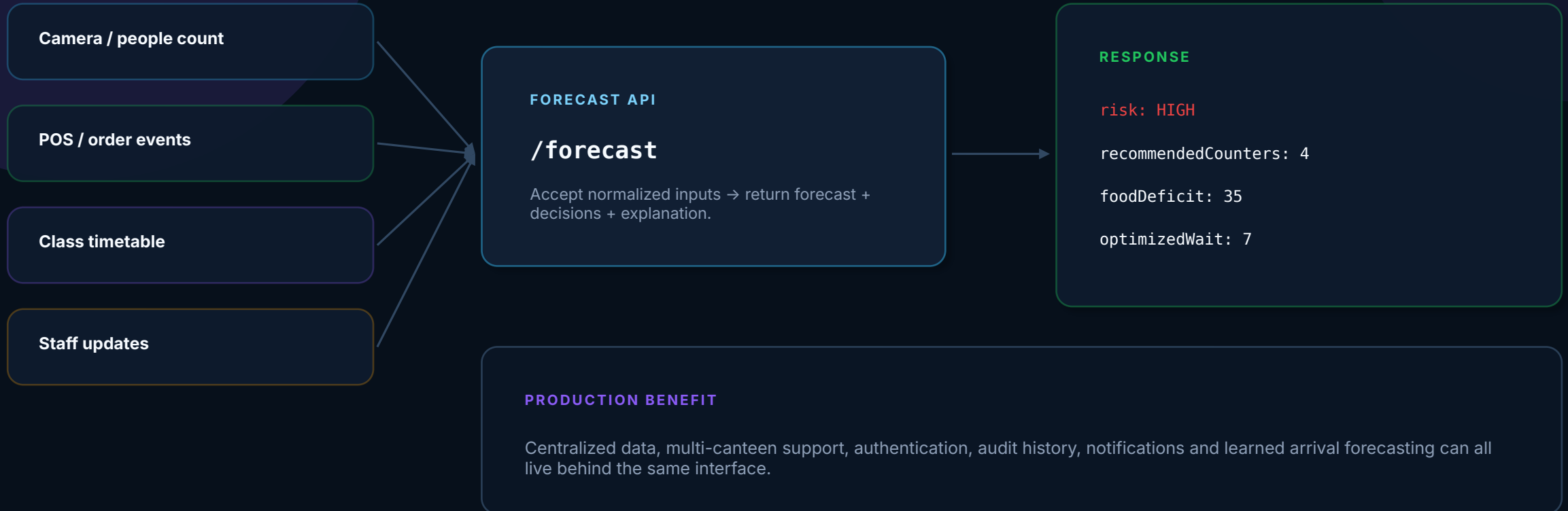
No external API required

The demo runs offline after download.

Backend-ready: the local `simulate()` function can later be replaced with a REST/API call to Python (Flask/FastAPI), Node.js or another service while keeping the same UI.

How the prototype can become a production system

A real deployment can replace manual inputs with live data sources and move forecasting to a backend service.



Rush hour - exact live-model calculation

This example follows the equations used by the working code, not the static placeholder values in the first prototype.

INPUTS

Current queue	70
Expected arrivals	45
Food available	80
Service rate	2.5 / min / counter
Active counters	2

CALCULATIONS

Total load	115
Current capacity	75
Over capacity	53%
One-counter capacity	37.5
Required counters	4

OUTPUT

HIGH RISK

4

recommended counters

+35

extra portions

14 → 7

wait time (min)

Action: open +2 counters and prepare 35 more portions.

Expected operational benefits

These are the benefits the system is designed to create. Quantitative improvement must be validated through a real canteen pilot.

01

Shorter queues

Open counters before congestion becomes severe.

02

Better staffing

Match active counters to demand instead of a fixed schedule.

03

Lower stock risk

Prepare the next batch based on projected demand.

04

Clearer decisions

Provide staff with a direct action, not only a chart.

05

Auditability

Explain why each alert and recommendation was generated.

06

Scalability

Use the same logic across canteens once backend data is connected.

Pilot KPI examples: wait time • queue length • stock-outs • food waste • counter utilization

LIMITATIONS

What the prototype does not yet do

Being explicit about the current boundaries makes the project easier to defend during a presentation or viva.

No trained ML arrival model

Expected arrivals are an input in the demo; the system does not learn from historical data yet.

Manual data collection

No live camera, POS or sensor connection is currently required or implemented.

Simplified queue assumptions

Service rate is treated as stable and students are modeled as one common queue.

Local-only storage

History is browser localStorage, not a shared institutional database.

No authentication / roles

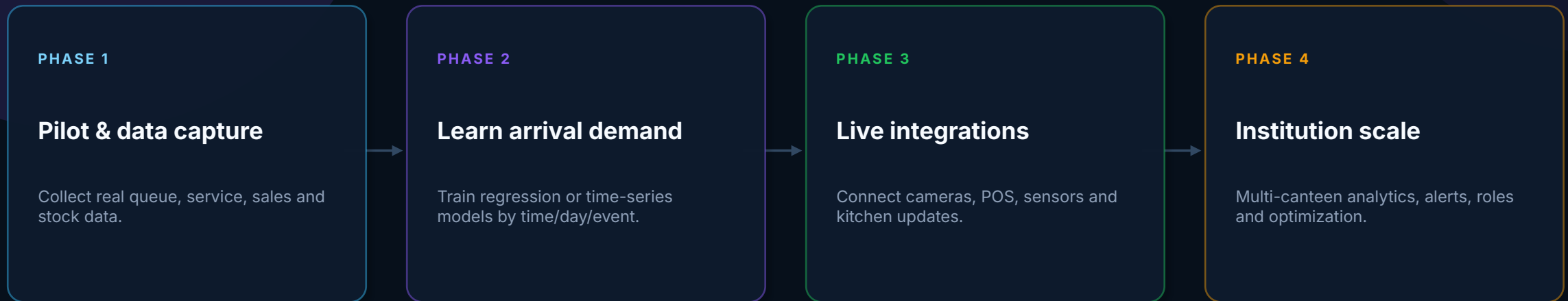
The standalone demo has no admin, staff or multi-canteen account system.

Impact not yet validated

Any percentage improvement should be treated as a target until measured in a pilot.

How QueueMind can evolve from demo to deployment

The architecture allows intelligence to improve in stages without rebuilding the interface.



Key principle: keep the decision engine explainable even when the arrival forecast becomes machine-learned.

From reactive canteen management to predictive operations.

The prototype already demonstrates the complete decision loop: capture inputs → forecast pressure → recommend staffing and stock → explain the reasoning → save the result.

Working model

Explainable

Backend-ready

PRESENTATION TAKEAWAY

- 01 Predict the next 15 minutes
- 02 Act before the queue peaks
- 03 Prepare the right amount of food
- 04 Show the reason behind every decision

Thank you

QueueMind - Canteen Flow Intelligence